ML and DL Research Career Roadmap

A working plan for moving from bootcamp projects to research and developer roles at AI labs, in India and abroad.

---

1. Where you stand today

You finished a broad machine learning, NLP and MLOps bootcamp and built about twenty nine repositories. That is real, consistent output and it proves you can finish things, which most beginners never manage. Looking at the list honestly, most of the work sits in the tutorial and Kaggle style category: house price prediction, bike price prediction, heart disease prediction, Fashion MNIST, IMDB sentiment, ANN basics, Netflix and movie data analysis. These are excellent for learning the mechanics of the field. Clear Path, RAG and the F1 pit stop project show you already reaching toward more original problems.

If the goal is a research or research adjacent role at a serious lab, the bar is not the number of repositories, it is depth, originality and evidence that you can identify a real problem, form a hypothesis and test it rigorously. Right now your GitHub reads as a strong learner's portfolio, not yet a researcher's portfolio. That is completely normal at this stage and it is very fixable.

2. What a lab is actually looking for

When labs such as Google DeepMind, FAIR at Meta, Microsoft Research, or a strong Indian group like Microsoft Research India, Adobe Research India, or a good IIT or IISc lab look at a junior candidate, they are checking for four things.

- Command of the mathematical and algorithmic fundamentals, not just library calls.
- Evidence of independent thinking: a question nobody assigned you, that you tried to answer yourself.
- Ability to read, reproduce and critique papers, not only use pretrained models.
- Public proof: a paper, a strong open source contribution, a blog post other researchers reference, or a hard to fake competition result.

3. Fix the fundamentals first

Core math and theory to lock in before going further:

- Linear algebra: Gilbert Strang's MIT 18.06 lectures.
- Probability and statistics, applied to estimation and inference, not just formulas.
- Calculus and optimization intuition: gradient descent, convexity, why training loops behave the way they do.

- Information theory basics: entropy, KL divergence, cross entropy. This explains half of every deep learning loss function.

Courses and resources worth your time:

- Stanford CS229 for math heavy machine learning.
- Stanford CS231n for vision, CS224n for NLP.
- fast.ai practical deep learning for a fast, applied counterweight to the theory.
- Andrej Karpathy's neural networks zero to hero series. Especially valuable right after a bootcamp, since it forces you to understand what is happening inside a framework instead of only calling fit and predict.
- Deep Learning by Goodfellow, Bengio and Courville for theory. Dive into Deep Learning for hands on practice.

4. Rebuild the portfolio around depth

This is the single most important shift. Stop starting new tutorial style projects and go deep on fewer things.

- Pick three to five landmark or recent papers and reimplement them from scratch in PyTorch, without copying existing repos. Good starting set: a small Transformer from Attention Is All You Need, a from scratch GPT following Karpathy's nanoGPT, LoRA fine tuning implemented manually, a small diffusion model on MNIST or CIFAR, and a retrieval augmented generation pipeline with your own evaluation metrics rather than library glue code. That last one builds directly on your existing RAG repo, so push it further instead of starting a new one.
- For every reimplementation, write up what you learned, what broke, what the paper left out, and how your numbers compare to the paper's reported numbers. That write up is worth more to a hiring researcher than the code itself.
- Contribute to one real open source project used by researchers: Hugging Face transformers, PyTorch, timm, or a smaller but active research codebase. Start with documentation or small bug fixes, then move toward feature contributions. A merged pull request in a serious repository is a strong signal.
- Enter two or three Kaggle or Zindi competitions seriously, not to win, but to learn how experienced practitioners approach a leaderboard and to network through the discussion forums.
- Choose one project that is entirely yours, ideally tied to a domain you already touched such as traffic systems from Clear Path or motorsport data from the F1 pit stop project, and take it from a notebook to a small research style study with a baseline comparison, an ablation table, and a clear positive or negative result. This becomes your flagship project.

5. Start writing and publishing

Blog: start a technical blog, either a personal site or a platform researchers actually read, and write one deep technical post a month. Cover paper explainers, your reimplementation notes, and honest write ups of failed experiments. A well written paper explainer earns more researcher attention than another prediction notebook.

Preprints: once you have a flagship result, write it up in a short paper format, four to eight pages, using a NeurIPS or ACL style template, and post it on arXiv in the cs.LG or cs.CL category. Very new accounts sometimes need an endorsement, which a professor or mentor can provide.

Workshops: target workshop tracks at NeurIPS, ICML and ICLR, and at ACL or EMNLP if your work is language focused. Workshop papers have a much lower bar than the main conference and are a completely legitimate way for an independent researcher to get a real, peer reviewed publication.

India specific venues: ICVGIP, COMAD, student symposia at IIT and IISc, and outreach programs run by CVIT at IIIT Hyderabad and the Robert Bosch Centre for Cyber Physical Systems at IISc.

Co-authoring: email professors at IIT, IISc, IIIT Hyderabad or IIIT Bangalore whose papers you actually read and enjoyed, with a short specific note about their work and an offer to help with implementation or experiments. Most professors respond well to a motivated student who already has something concrete to point to on GitHub.

6. Presence where researchers actually are

X, formerly Twitter, is where most ML research discourse happens in real time. Follow and engage with researchers whose papers you are reading. Comment with real technical questions rather than generic praise, and post your own reimplementation threads. People like Archie Sengupta, who you already follow, are a good model of how technical people build an audience by consistently sharing real work rather than only opinions.

LinkedIn: keep it updated as a professional record and post your flagship project and paper once ready, but treat it as secondary to X and GitHub for actual research visibility.

Reddit: r slash MachineLearning for paper discussion and hiring threads, r slash LearnMachineLearning and r slash deeplearning for early stage help. r slash MachineLearning has a high bar for self promotion posts, so contribute genuine discussion before posting your own work.

GitHub: clean up your existing repositories. Add clear README files with problem statement, method, results and what you would do differently. Archive or make private the very early tutorial style repos, and pin the three or four that best represent where you are heading now.

7. The roadmap in stages

Stage one, months zero to six

Fundamentals through CS229 and Karpathy's series. Two paper reimplementations pushed to GitHub with proper write ups. One merged pull request to a real open source ML project. Start the blog. Apply to Google Summer of Code, MITACS Globalink if you are considering Canada, and email IIT or IISc labs directly for a summer research position.

Stage two, months six to eighteen

Finish your flagship project and submit it as a workshop paper or arXiv preprint. Apply for research internships at Microsoft Research India, Adobe Research India, Google Research India, Qualcomm AI Research, or Wadhwani AI. Seriously evaluate whether your next formal step is a master's by research at IISc or a strong IIT, or a direct research engineer role, since almost every deep research path eventually needs a master's, a PhD, or a genuinely strong independent publication record as a substitute.

Stage three, months eighteen to thirty six

With one or two real publications or a strong applied research track record, apply to residency style programs abroad such as the OpenAI Residency or Google's research training programs when they are open, to the Allen Institute for AI in Seattle, MILA in Montreal, or Vector Institute in Toronto, or apply directly to PhD programs, which remain one of the most reliable long term paths into frontier AI research.

8. Where to apply, a working list

In India

- Microsoft Research India, Bangalore
- Adobe Research India, Bangalore and Noida
- Google Research India, Bangalore
- Qualcomm AI Research, Hyderabad and Bangalore
- NVIDIA India research and engineering roles
- Wadhwani AI, Mumbai
- IISc CSA department and the Robert Bosch Centre for Cyber Physical Systems
- IIT Bombay, Delhi, Madras and Kanpur AI and NLP labs
- IIIT Hyderabad, CVIT and LTRC groups
- Ola Krutrim and Sarvam AI for Indian foundation model work

Abroad

- Google DeepMind
- FAIR at Meta
- Microsoft Research, Redmond and Cambridge
- Allen Institute for AI
- MILA, Montreal
- Vector Institute, Toronto
- Max Planck Institute for Intelligent Systems
- University labs at Stanford, CMU, Berkeley and Oxford, for the PhD route

Check current openings directly on each lab's careers page, since residency and internship programs open and close on a rolling basis and details shift year to year.

9. A note on realism and pacing

This is a multi year path for almost everyone who does not come from a top tier CS undergraduate program with early research exposure, and that is completely normal. The single biggest lever you have right now is depth. Stop starting new tutorial style projects. Take one thing you have already built, RAG or Clear Path are good candidates, and go three levels deeper than you have gone before. That one act, repeated four or five times over the next year, will do more for your research career than another twenty notebooks.

Quick checklist: fundamentals through CS229 and Karpathy. Three to five paper reimplementations with written analysis. One merged open source pull request. A flagship project with real ablations. A monthly technical blog. One workshop paper or arXiv preprint within eighteen months. Active, substantive presence on X and GitHub. Direct outreach to two or three professors or labs every month.